DiskANN cpp_main

论文复现实验框架设计

Partition / Build Local Graph / Merge Graph

修订完整版

> **文档定位：** 面向 Microsoft DiskANN 旧版 C++ cpp_main 分支，构建模块可插拔、论文方案可复现、模块组合可验证、Agent 可自动执行的长期实验工程。

版本日期：2026-07-31

技术基线：microsoft/DiskANN · cpp_main · C++

修订说明

本版本在原有“公共流水线规范 + 原始 DiskANN Baseline + 单篇论文独立 Feature + Harness 组合配置”结构上直接修订，不削减总体设计复杂度。修订重点是使模块契约与 cpp_main 分支的真实构建行为一致，并补齐 Agent 自动化实验所需的 Artifact、Manifest、Evaluation、兼容性和复现模式。

修订项	原表述中的问题	修订后的原则
技术基线	未明确旧版 C++ 与新 Rust 主线的边界	明确固定 microsoft/DiskANN cpp_main 分支及具体 commit
节点编号	局部图被要求直接使用全局 ID	局部图使用 LocalNodeId，并强制提供 LocalToGlobalIdMap
Baseline Merge	将重新剪枝、连通性修复写成原始行为	准确限定为映射、聚合、去重、随机打乱与截断
产物模型	图产物与可达率、Recall 等评估指标混合	Artifact、BuildManifest、EvaluationReport 三层分离
可复现性	统一要求字节级确定性	区分 upstream faithful、instrumented、deterministic experimental
通用约束	单调性等论文性质被设为所有实现硬约束	只保留结构不变量；论文性质进入 paper-specific checks

> **核心结论：**完整架构是合理的。应保留 Spec Kit、模块 Registry、Harness、Loop Engine、requires/provides 和组合实验机制；需要修改的是准确性，而不是将方案简化为一次性脚本。

目录

1. 文档目标与技术边界 4

2. 三个核心模块的科学问题与定位 5

3. 顶层设计原则与稳定不变量 6

4. SDD 与 Spec Kit 文档层次 7

5. 核心流水线与公共执行模型 9

6. 节点编号与重叠节点模型 10

7. Artifact、Manifest 与 Evaluation 分层 12

8. Partition 模块契约 13

9. Build Local Graph 模块契约 15

10. Merge Graph 模块契约.....16
11. cpp_main 原始 Baseline 的精确定义.....17
12. 单篇论文 Feature 的规范模板.....19
13. 模块兼容性与能力声明.....22
14. 实验组合配置与公平对比.....23
15. 可复现性模式与有效运行判定.....25
16. Harness 与 Loop Engine.....26
17. 推荐目录结构与实施顺序.....27
18. Spec Kit 的实际使用方式.....29
19. 完整验收清单与最终判断.....31
附录 A-C.....32

1. 文档目标与技术边界

1.1 目标

本项目的目标不是只复现某一篇论文，也不是只为某一次实验编写脚本，而是建立一套长期可扩展的 DiskANN 论文复现实验工程。该工程应支持在不覆盖原始实现的前提下，对 Partition、Build Local Graph 和 Merge Graph 等模块进行独立替换、耦合替换、跨论文组合和统一评估。

- 保留并冻结原始 DiskANN Baseline。
- 以稳定公共契约接入不同论文实现。
- 保证不同实现可以复用阶段性产物，避免无条件全流程重跑。
- 支持 Agent 根据 SDD、任务清单和反馈循环完成实现、验证与实验。
- 支持单变量消融、强依赖组合和跨论文模块组合。

1.2 明确的代码基线

本框架只以 Microsoft DiskANN 旧版 C++ cpp_main 分支为代码基线。所有 Baseline 结果、论文适配和接口映射都必须绑定到一个固定 commit；不得使用“当前 cpp_main”作为不可追踪的浮动基线。

基线身份配置

```
baseline:
  project: microsoft/DiskANN
  branch: cpp_main
  commit: <required-pinned-commit>
  language: C++
  target_pipeline: build_merged_vamana_index
  build_mode: static_offline_build
  source_modified: false
```

> **强制要求：**未记录 commit、编译参数、工作区修改状态和数据集校验值的运行，不得标记为有效 Baseline 或有效论文复现结果。

1.3 研究范围

第一阶段的核心研究范围是 cpp_main 中大规模内存受限 Vamana 构建路径的三个阶段：Partition、Build Local Graph 和 Merge Graph。Search、Disk Layout、Memory Cache、Prefetch 与 I/O Scheduling 可作为后续扩展阶段，但核心框架不能把模块分类永久限制为三个。

```
touched_stages:
  - partition
```

```
  - local_graph_build
  - global_merge

optional_future_stages:
  - graph_pruning
  - global_refinement
  - disk_layout
  - node_reordering
  - memory_cache
  - prefetch
  - search_traversal
```

1.4 非目标

- 第一版本不实现构图完成后的在线插入、删除和更新。
- 第一版本不把 Qdrant 集成与论文忠实复现混入同一个 Feature。
- 框架不要求所有论文都采用 Vamana、RobustPrune、重叠分块或同一种 Merge 语义。
- 框架不自动宣称任何适配实现属于 Exact reproduction。

2. 三个核心模块的科学问题与定位

2.1 Partition

为了解决的科学问题：在单机内存无法容纳全量向量数据及图构建中间状态的条件下，如何将大规模高维向量数据划分为若干规模相对均衡、能够独立进行局部构图的子集，同时尽量保持原始数据中的近邻关系，并控制重叠数据带来的额外存储、构建和合并开销。

目的与作用：将全局构图转换为多个受内存预算约束的局部构图任务，为并行或分布式执行提供任务边界；通过聚类、重叠分配、边界节点复制或论文特定策略，提高真实近邻共同出现在至少一个分块中的概率。

公共规范只要求 Partition 产物能够被下游消费，并提供完整的局部到全局节点映射。是否重叠、如何选择边界点、是否输出质心和副本所有权，属于具体算法或可选能力。

2.2 Build Local Graph

为了解决的科学问题：在每个分块只能获得局部数据与有限候选邻居的条件下，如何在出度、内存和构建时间约束下生成高质量局部图，使其兼具局部近邻准确性、邻边多样性和有效导航能力，并为后续 Merge 提供可复用的局部拓扑。

目的与作用：独立构建各分块的局部图，保存局部邻接关系、局部入口点和 LocalNodeId 到 GlobalNodeId 的映射。原始 Baseline 使用 cpp_main 的 Index::build 和低出度参数；论文实现可以替换候选生成、剪枝、插边、构图次序或整个局部图算法。

单调性、特定剪枝定理、block-aware 性质等不应成为所有实现的统一接口约束，而应作为论文特定质量验证。

2.3 Merge Graph

为了解决的科学问题：在各分块局部图独立构建完成、同一全局节点可能以不同局部编号出现在多个重叠分块中的条件下，如何正确完成局部节点到全局节点的映射，聚合同一全局节点在不同分块中的候选邻接关系，并在不执行全量重新构图的前提下控制重复边、出度和中间状态开销，生成可在全量数据范围内统一搜索的全局图。

Baseline Merge 的目的：完成 LocalNodeId 到 GlobalNodeId 的转换，聚合同一全局节点在多个分块中的局部邻接表，进行候选去重和出度控制，并输出全局编号空间下的图索引。

增强型 Merge 的研究空间：论文实现可以增加跨分块候选发现、距离重计算、统一剪枝、全局 refinement、连通性检测与修复。这些是可选增强能力，不是 cpp_main 原始 merge_shards 已经提供的功能。

> **准确性边界：**不得把“研究上希望 Merge 达到的能力”写成“原始 DiskANN Baseline Merge 的实际行为”。Baseline 语义必须与固定 commit 中的 merge_shards 一致。

3. 顶层设计原则与稳定不变量

3.1 与论文无关的稳定约束

- 每个原始向量具有唯一且稳定的 GlobalNodeId。
- 每个分块可以拥有独立、连续的 LocalNodeId 空间。
- 每个局部产物必须提供完整的 LocalNodeId → GlobalNodeId 映射。
- 同一 GlobalNodeId 可在多个重叠分块中拥有不同 LocalNodeId。
- 最终全局图必须使用 GlobalNodeId 空间，且唯一逻辑节点数等于原始数据集节点数。
- 每个阶段产物必须版本化、可校验、可追踪并可被独立复用。
- 模块实现必须声明输入需求、输出能力和不支持的情况。
- Baseline 和论文方案的对比必须使用明确的公平性协议。

3.2 接口约束与算法实现分离

核心规范应规定	核心规范不应固定
模块接收什么输入	必须采用哪一种分块算法
模块输出什么产物	必须使用 RobustPrune
节点编号与映射规则	必须通过重叠节点建立跨块边
哪些字段是必选或可选	必须在 Merge 阶段执行连通性修复
哪些结构不变量必须成立	必须满足某篇论文的单调性定义
如何验证产物和结果	必须以某一种文件布局实现

这样可以同时兼容：不重叠分块、重叠分块、使用 kNN 图的分块、不同局部剪枝方法、无传统 Merge 的全局 refinement，以及同时修改多个阶段的论文。

3.3 原始代码不覆盖

Baseline 和所有论文实现必须同时存在，通过 Registry 与实验配置选择实现。禁止以“复现下一篇论文”为理由覆盖前一篇论文或原始 DiskANN 的代码。

```
partition_impl: paper_a_partition
local_build_impl: diskann_cpp_main_baseline
merge_impl: paper_c_merge
```

该原则用于保证可回退、可消融、可组合和可复现。

3.4 Baseline Parity Gate

在任何论文替换实验开始前，必须验证“原生 cpp_main 命令直接运行”与“经统一 Pipeline/Adapter/Harness 运行”在统计意义上保持一致。

- 节点数量、图文件可解析性、最大/平均出度。
- 局部与全局入口点或 medoid。
- Recall、构建时间、峰值内存和查询性能。
- 允许原始随机行为导致的波动，但不得存在系统性偏差。

4. SDD 与 Spec Kit 文档层次

总体规范拆分为四层。四层分别管理公共不变量、原始行为、论文实现和实验组合，避免将所有论文和所有参数持续写入一个 spec.md。

4.1 第一层：核心流水线规范

```
specs/
└── 000-modular-cpp-diskann-pipeline/
    ├── spec.md
    ├── plan.md
    ├── data-model.md
    ├── research.md
    ├── contracts/
    │   ├── identifiers.md
    │   ├── artifact-ref.md
    │   ├── partition-artifact.md
    │   ├── local-graph-artifact.md
    │   ├── global-graph-artifact.md
    │   ├── build-manifest.md
    │   ├── evaluation-report.md
    │   └── capabilities.md
    └── tasks.md
```

该层只定义公共接口、编号模型、产物契约、能力声明、执行状态和验证规则，不固定某篇论文的内部算法。

4.2 第二层：cpp_main 原始 Baseline

```
specs/
└── 001-cpp-main-upstream-baseline/
    ├── spec.md
    ├── plan.md
    ├── research.md
    ├── source-mapping.md
    ├── experiment-spec.md
    ├── parity-checklist.md
    └── tasks.md
```

该 Feature 冻结仓库、分支、commit、构建工具链、原始命令、参数、阶段产物与结果。它必须准确描述 build_merged_vamana_index、partition_with_ram_budget、Index::build 和 merge_shards，而不能用理想化算法描述代替源码事实。

4.3 第三层：每篇论文的独立 Feature

```
specs/
├── 010-paper-a-partition/
├── 020-paper-b-local-build/
├── 030-paper-c-merge/
└── 040-paper-d-coupled-partition-merge/
```

论文只修改一个模块时，使用单模块 Feature；论文同时修改两个或多个强依赖阶段时，可以建立一个耦合 Feature，但必须分别声明各阶段的接口变化和依赖。

4.4 第四层：实验组合配置

不同模块组合不重新生成 SDD，而由 Harness 配置管理。SDD 定义实现和契约，实验配置选择实现、参数、数据集、资源预算和评估规则。

```
Baseline = P0 + B0 + M0
实验一   = P1 + B0 + M0
实验二   = P0 + B1 + M0
实验三   = P0 + B0 + M1
实验四   = P1 + B1 + M0
实验五   = P1 + B1 + M1
```

5. 核心流水线与公共执行模型

端到端逻辑流程

```
DatasetRef
    ↓
PartitionStrategy
    ↓  PartitionArtifact + BuildManifest
LocalGraphBuilder × N
    ↓  LocalGraphArtifact[] + BuildManifest[]
GraphMerger
    ↓  GlobalGraphArtifact + BuildManifest
Evaluator
    ↓  EvaluationReport
```

5.1 公共引用对象

C++ 逻辑接口

```
struct ArtifactRef {
    std::filesystem::path manifest_path;
    std::string artifact_type;
    std::string format_version;
    std::string checksum;
};

struct StageResult {
    ArtifactRef artifact;
    ArtifactRef build_manifest;
    ArtifactRef stage_report;
```

```
};
```

接口传递 ArtifactRef，而不是要求将全量图、所有分块或全部邻接表加载为内存对象。ArtifactRef 指向版本化 Manifest，Manifest 再引用具体文件。

5.2 StageContext

```
struct StageContext {
    WorkspaceRef workspace;
    uint64_t memory_budget_bytes;
    uint32_t thread_budget;
    TemporaryStorageRef temporary_storage;
    VectorAccessor* vector_accessor;
    DistanceFunction* distance_function;
    CheckpointStore* checkpoint_store;
    MetricsSink* metrics_sink;
    CancellationToken* cancellation_token;
};
```

需要重新计算距离、访问原始向量、创建临时文件或写入检查点的论文实现，通过 StageContext 获取能力。模块必须在 capabilities 中声明这些需求。

5.3 阶段状态

```
enum class StageStatus {
    PLANNED,
    RUNNING,
    SUCCEEDED,
    FAILED,
    CANCELLED,
    INVALID_OUTPUT,
    REUSED
};
```

失败或不完整产物不能被标记为成功；只有通过契约校验的 Artifact 才能进入下游。

6. 节点编号与重叠节点模型

6.1 三类编号

```
using GlobalNodeId = uint32_t;
using LocalNodeId  = uint32_t;
using PartitionId  = uint32_t;
```

编号	作用	约束
GlobalNodeId	原始数据集中向量的稳定逻辑编号	0 ⩽ id < global_node_count；全流程稳定
LocalNodeId	节点在单个分块中的物理编号	仅在对应 Partition 内有效；通常连续
PartitionId	标识一个分块及其阶段产物	在一次构建任务中唯一

6.2 LocalToGlobalIdMap

每个分块必须保存完整映射：

```
LocalToGlobalIdMap[LocalNodeId] = GlobalNodeId
```

局部图邻接表默认使用 LocalNodeId。Merge 通过 LocalToGlobalIdMap 将局部节点和局部邻居转换为全局节点。该模型与 cpp_main 的 _ids_uint32.bin 语义一致。

6.3 重叠节点

```
Partition 0: LocalNodeId 17  ──┐
                               ├──> GlobalNodeId 9245
Partition 3: LocalNodeId 211 ──┘
```

同一 GlobalNodeId 在不同分块中可以拥有不同 LocalNodeId。公共契约禁止将“相同全局节点必须保留相同局部编号”作为要求。

6.4 编号相关硬不变量

- LocalToGlobalIdMap 的长度必须等于 local_node_count。
- LocalToGlobalIdMap 中每个值必须落入全局节点范围。
- 局部邻接表中的每个邻居必须落入对应分块的 LocalNodeId 范围。
- 最终全局图中不能残留 LocalNodeId 或未解析副本节点。
- 最终唯一 GlobalNodeId 数量必须等于数据集节点数量。

7. Artifact、Manifest 与 Evaluation 分层

7.1 Artifact

Artifact 是不可变的数据产物，例如分块向量文件、LocalToGlobal 映射、局部图、全局图、质心文件、边界节点文件和论文专有元数据。Artifact 只描述“数据是什么”，不混入运行时统计结论。

7.2 BuildManifest

```
BuildManifest
 ├─── run_id
 ├─── stage_name
 ├─── implementation_id
 ├─── repository / branch / commit
 ├─── dirty_worktree
 ├─── compiler / build_flags
 ├─── dataset_ref / checksum
 ├─── parameters
 ├─── input_artifacts[]
 ├─── output_artifacts[]
 ├─── start_time / end_time
 ├─── peak_rss_bytes
 ├─── temporary_disk_bytes
 ├─── host_fingerprint
 └─── status / failure_reason
```

BuildManifest 描述“如何生成 Artifact”，用于追溯和重放。

7.3 EvaluationReport

```
EvaluationReport
 ├─── graph_integrity
 ├─── unique_node_count
 ├─── invalid_neighbor_count
 ├─── degree_distribution
 ├─── reachable_node_ratio
 ├─── connected_component_statistics
 ├─── recall_at_k
 ├─── qps
 ├─── latency_p50_p95_p99
 ├─── distance_computations
 ├─── search_hops
 ├─── storage_io_metrics
 └─── comparison_summary
```

可达率、Recall、延迟和搜索跳数属于 EvaluationReport，不属于 GlobalGraphArtifact 的固有字段。

7.4 分层后的判断流程

```
Artifact 是否生成
    ↓
Contract Validator：产物是否合法
    ↓
Evaluator：图质量与性能是否达标
    ↓
Comparator：相对 Baseline 或论文结果如何
    ↓
Loop Engine：继续、重试、终止或进入下一配置
```

8. Partition 模块契约

8.1 逻辑接口

```
class PartitionStrategy {
public:
    virtual StageResult run(
        const DatasetRef& dataset,
        const PartitionConfig& config,
        StageContext& context
    ) = 0;
};
```

8.2 PartitionArtifact

```
PartitionArtifact
├── format_version
├── dataset_ref
├── dataset_checksum
├── global_node_count
├── vector_dimension
├── distance_metric
├── partition_count
├── algorithm_descriptor
├── partition_manifests[]
│   ├── partition_id
│   ├── local_node_count
│   ├── shard_vector_file_ref
│   ├── local_to_global_id_map_ref
│   ├── local_to_global_map_checksum
│   └── optional_metadata_refs[]
├── optional_global_to_partitions_index_ref
├── optional_replica_metadata_ref
├── optional_centroid_metadata_ref
```

```
├── capabilities
└── integrity_checksum
```

8.3 必选字段与可选能力

类别	字段或能力	说明
必选	local_to_global_id_map_ref	所有 Partition 实现都必须输出
必选	partition_id / local_node_count	定义分块身份和局部编号范围
必选	dataset_checksum	防止阶段产物引用错误数据集
可选	supports_overlap	是否允许同一 GlobalNodeId 出现在多个分块
可选	provides_boundary_metadata	是否显式输出边界节点
可选	provides_centroids	是否输出聚类质心
可选	provides_replica_ownership	是否区分主副本与复制副本

示例 capability 声明

```
capabilities:
  produces_local_to_global_map: true
  supports_overlap: true
  provides_boundary_metadata: false
  provides_replica_metadata: false
  provides_replica_ownership: false
  provides_centroids: true
  guarantees_partition_size_limit: false
```

8.4 Partition 验证指标

- 分块规模分布与最大/平均规模比。
- 真实 kNN 对共同出现在至少一个分块中的比例。
- 平均复制因子和最大复制次数。
- 阶段峰值内存、临时空间和运行时间。
- LocalToGlobal 映射完整性和全局节点覆盖率。

这些是 Evaluation 指标，不应要求所有算法以相同方式达到。每篇论文在 experiment-spec.md 中设置阈值和对比基线。

9. Build Local Graph 模块契约

9.1 逻辑接口

```
class LocalGraphBuilder {
public:
    virtual StageResult build(
        const ArtifactRef& partition_shard,
        const LocalBuildConfig& config,
        StageContext& context
    ) = 0;
};
```

9.2 LocalGraphArtifact

```
LocalGraphArtifact
├─── format_version
├─── partition_id
├─── local_node_count
├─── id_space: local
├─── graph_file_ref
├─── local_to_global_id_map_ref
├─── local_entry_point
├─── declared_max_degree
├─── observed_max_degree
├─── source_partition_checksum
├─── algorithm_descriptor
├─── capabilities
└─── integrity_checksum
```

9.3 LocalGraphView

```
class LocalGraphView {
public:
    virtual std::span<const LocalNodeId>
        local_neighbors(LocalNodeId node) const = 0;

    virtual GlobalNodeId
        to_global(LocalNodeId node) const = 0;
};
```

统一框架提供逻辑读取视图，但不强制改写 Baseline 的原始文件格式。这样既能忠实接入 cpp_main，又允许论文仓库保留自己的局部编号与存储布局。

9.4 通用硬约束与论文特定性质

通用硬约束	论文特定验证
局部节点编号合法	单调路径性质
邻接节点存在	特定 RobustPrune 或 occlusion 条件
图文件可解析	block-aware 邻边选择
出度不超过声明上限	方向多样性或理论覆盖性质
映射与源分块一致	论文定义的局部导航保证

核心契约不要求一定调用 RobustPrune。原始 Baseline 使用其自身 Index::build 路径；论文实现可替换候选生成、剪枝或整个局部图构建过程。

10. Merge Graph 模块契约

10.1 逻辑接口

```
class GraphMerger {
public:
    virtual StageResult merge(
        const ArtifactRef& partition_manifest,
        std::span<const ArtifactRef> local_graphs,
        const MergeConfig& config,
        StageContext& context
    ) = 0;
};
```

10.2 GlobalGraphArtifact

```
GlobalGraphArtifact
 ├─── format_version
 ├─── global_node_count
 ├─── id_space: global
 ├─── graph_file_ref
 ├─── global_entry_points_ref
 ├─── declared_max_degree
 ├─── observed_max_degree
 ├─── source_partition_artifact
 ├─── source_local_graph_artifacts[]
 ├─── algorithm_descriptor
 ├─── capabilities
 └─── integrity_checksum
```

10.3 Merge 的公共不变量

- 所有 LocalNodeId 必须在输出前转换为 GlobalNodeId。
- 最终全局图的逻辑节点数量必须等于原始数据集节点数量。
- 不得存在越界邻居、未解析副本或非法 ID。
- 输出图的实际出度不得超过实现声明的最大出度。
- 输出产物必须可被统一查询器或 Baseline 查询路径加载。

10.4 可选增强能力

```
merge_capabilities:
  supports_candidate_union: true
  supports_local_to_global_remap: true
  supports_cross_partition_search: optional
  supports_distance_recomputation: optional
  supports_distance_based_pruning: optional
  supports_block_aware_pruning: optional
  supports_connectivity_detection: optional
  supports_connectivity_repair: optional
  supports_global_refinement: optional
```

可达性检测和修复可以是论文实现或独立后处理模块的能力，但不能被写成 cpp_main Baseline Merge 的既有行为。

11. cpp_main 原始 Baseline 的精确定义

11.1 Baseline 总流程

当全量内存索引估算能够放入给定 RAM budget 时，build_merged_vamana_index 直接执行一次 Index::build。只有在全量图无法放入预算时，才进入分块构建和合并路径。

```
if full_index_ram < ram_budget:
    IndexWriteParameters(L, R)
    Index::build(base_file)
    Index::save(mem_index_path)
else:
    P0 = partition_with_ram_budget(...)
    B0 = Index::build(each shard, degree = 2R/3)
    M0 = merge_shards(..., max_degree = R)
```

代码依据：microsoft/DiskANN, cpp_main, src/disk_utils.cpp，build_merged_vamana_index。

11.2 P0：Baseline Partition

内存受限路径调用 partition_with_ram_budget，并将 graph_degree 传为 2R/3，将 k_base 传为 2。P0 生成分块 ID 文件和质心文件，随后框架根据每个 _ids_uint32.bin 从原始数据中提取 shard 数据。

```
partition_with_ram_budget<T>(
    base_file,
    sampling_rate,
    ram_budget,
    2 * R / 3,
    merged_index_prefix,
    2              // k_base
);
```

代码依据：cpp_main/src/disk_utils.cpp，partition_with_ram_budget 调用；cpp_main/include/partition.h，函数声明。

> **注意：** P0 的 “k_base=2” 是固定 commit 中的 Baseline 行为，不应被提升为所有论文 Partition 的公共要求。

11.3 B0：Baseline Local Build

每个分块使用 IndexWriteParametersBuilder(L, 2R/3)，with_saturate_graph(false)，然后调用 Index::build 或 build_filtered_index，并保存为局部内存图。局部图使用 shard 内部的局部编号；对应 _ids_uint32.bin 提供局部到全局映射。

```
IndexWriteParametersBuilder(L, 2 * R / 3)
    .with_filter_list_size(Lf)
    .with_saturate_graph(false)
    .with_num_threads(num_threads)
    .build();

_index.build(shard_base_file, shard_base_pts);
_index.save(shard_index_file);
```

代码依据：cpp_main/src/disk_utils.cpp，分块循环与局部 Index::build。

11.4 M0：Baseline Merge

M0 的实际语义必须按照 merge_shards 描述：

1. 读取每个 shard 的 LocalToGlobal ID map。
2. 根据所有 ID map 计算全局节点数量，并建立 GlobalNodeId → shard 的反向关系。
3. 读取并转换各 shard 的 medoid。

4. 对于同一 GlobalNodeId，读取其在所有相关 shard 中的局部邻接表。
5. 通过 idmaps[shard][local_neighbor] 将局部邻居转换为全局邻居。
6. 使用布尔集合去除重复候选邻居。
7. 对候选集合执行随机打乱。
8. 截断到 max_degree=R，并写入全局图。

```
baseline_merge:
  candidate_source: union_of_replica_local_adjacencies
  id_conversion: local_to_global
  duplicate_removal: true
  candidate_distance_recomputation: false
  distance_based_pruning: false
  candidate_selection: random_shuffle_then_truncate
  connectivity_detection: false
  connectivity_repair: false
  output_id_space: global
```

代码依据：cpp_main/src/disk_utils.cpp，merge_shards：read_idmap、node_shard、ID 重命名、std::shuffle 与 max_degree 截断。

> **禁止误述：** M0 不执行基于向量距离的统一 RobustPrune，也不显式检测或修复不可达子图。论文增强 Merge 可以实现这些能力，但必须使用独立 implementation_id。

11.5 Baseline 身份分层

Baseline 类型	允许修改	结果要求
Upstream Faithful	不修改语义；固定源码与构建环境	统计稳定，不要求图 checksum 完全一致
Instrumented	仅加入计时、内存、I/O、产物保留等观测	需通过与 Upstream 的 parity gate
Deterministic Experimental	允许固定随机种子等显式修改	必须记录偏差，并作为独立变体

12. 单篇论文 Feature 的规范模板

12.1 论文身份

paper-analysis.md

```
## Paper Identity
```

```
- Title:
- Authors:
- Venue:
- Year:
- Paper version / DOI:
- Official repository:
- Repository commit:
- Target DiskANN branch: cpp_main
- Target DiskANN commit:
- Reproduction owner:
- Reproduction date:
```

12.2 替换范围

```
## Replacement Scope

- [ ] Partition
- [ ] Build Local Graph
- [ ] Merge
- [ ] Graph Pruning
- [ ] Global Refinement
- [ ] Search
- [ ] Disk Layout
- [ ] Memory Cache
- [ ] Prefetch / I/O Scheduling
```

12.3 论文机制与源码映射

```
## Claimed Mechanism
1. 论文解决的问题:
2. 核心算法步骤:
3. 相对原始 DiskANN 的改变:
4. 论文声称的收益与适用条件:

## Source Mapping
| 论文算法步骤 | 官方源码位置 | 当前适配位置 | 验证方法 |
|---|---|---|---|
```

论文机制只描述论文实际提出的内容，不能混入为了接入统一框架而做的工程设计。

12.4 输入假设与输出保证

```
## Input Assumptions
- 是否要求预先存在 kNN 图:
- 是否要求聚类中心:
- 是否要求边界节点:
```

```
- 是否要求重叠副本或 replica ownership:
- 是否要求访问原始向量并重新计算距离:
- 接受的 ID 空间: local / global / both
- 是否依赖论文专有元数据:

## Output Guarantees
- 输出 Artifact 类型:
- 输出 ID 空间:
- 是否保留 LocalToGlobal 映射:
- 是否提供边界/质心/副本元数据:
- 是否保证出度上限:
- 是否可直接被 Baseline 下游消费:
```

12.5 复现类型

类型	定义	报告要求
Exact reproduction	严格按照论文与官方源码实现，关键步骤和参数语义保持一致	必须说明与原环境的差异仍不影响算法等价性
Faithful adaptation	核心算法保持一致，但为接入 cpp_main 修改接口、文件格式或工程组织	必须完整记录适配点与预期影响
Inspired implementation	只借鉴论文思想，不能声称等价复现	实验结论必须使用“受启发实现”表述

12.6 偏差记录

论文描述	当前实现	偏差原因	预期影响	验证方式
原论文数据集 A	SIFT100M	原数据不可获得	数据分布可能改变算法收益	增加第二数据集与敏感性分析
参数 X	参数 Y	受内存预算限制	图质量或构建时间变化	固定预算与固定参数两类比较
官方文件格式	统一 Artifact 格式	接入 Pipeline	理论上不改变算法语义	与官方输出做结构和结果对比

12.7 成功标准

- 算法步骤与论文/官方源码映射完整。
- 接口适配不改变论文关键语义，或偏差已明确记录。
- 在论文指定数据和参数可获得时，优先复现论文表格或趋势。
- 在统一 DiskANN 框架中通过契约校验和 Baseline 对比。
- 结论区分“论文复现是否成功”和“在当前环境中是否优于 Baseline”。

13. 模块兼容性与能力声明

13.1 requires / provides 模型

```
implementation: paper_c_merge
stage: global_merge

requires:
  artifact_types:
    - partition_artifact/v1
    - local_graph_artifact/v1
  id_space:
    local_graph: local
    output: global
  capabilities:
    - stable_global_node_id
    - local_to_global_map
    - boundary_node_metadata
    - partition_centroids
  runtime_services:
    - vector_accessor
    - distance_function

provides:
  artifact_types:
    - global_graph_artifact/v1
  capabilities:
    - global_graph
    - max_degree_enforcement
    - distance_based_pruning
    - connectivity_detection
```

13.2 运行前兼容性检查

```
Partition provides
        ↓
是否满足 Local Build requires
        ↓
Local Build provides
        ↓
是否满足 Merge requires
        ↓
Artifact format / ID space / version 是否兼容
        ↓
允许执行或在运行前拒绝
```

13.3 兼容与不兼容示例

上游实现	下游要求	判断
Paper A Partition 提供 boundary metadata	Paper B Merge 要求 boundary metadata	兼容
No-overlap Partition 不提供 replica metadata	Paper C Merge 强制依赖 replica ownership	不兼容，禁止运行
Baseline Local Graph 使用 local ID + map	Paper D Merge 接受 local ID + map	兼容
Paper E Local Graph 只输出 global ID	Baseline M0 只接受原始 shard 格式	需 Adapter，不能直接组合

13.4 Adapter 的位置

当论文官方产物格式与核心契约不同，但算法语义可保持时，应通过 Adapter 转换，不应直接修改论文核心算法。Adapter 也必须有独立版本、测试和偏差记录。

14. 实验组合配置与公平对比

14.1 完整实验配置示例

```
experiment_id: exp-paper-a-partition-sift100m

baseline:
  repository: microsoft/DiskANN
  branch: cpp_main
  commit: <pinned-commit>
  mode: instrumented

pipeline:
  partition:
    implementation: paper_a_partition
    config:
      num_partitions: 16
      overlap_ratio: 0.10

  local_graph:
    implementation: diskann_cpp_main_baseline
    config:
      L_build: 100
      local_degree: 42
      alpha: 1.2
      threads: 32

  merge:
    implementation: diskann_cpp_main_baseline
    config:
      max_degree: 64
```

```
resources:
  memory_budget_gb: 200
  temporary_disk_budget_gb: 1200
  cpu_threads: 32

reproducibility:
  mode: statistical
  repetitions: 5

validation:
  contract_checks: strict
  parity_checks: required

comparison:
  modes:
    - fixed_parameters
    - fixed_memory_budget
    - fixed_recall

dataset:
  name: sift100m
  checksum: <required>

evaluation:
  top_k: 10
  search_list: [50, 100, 150, 200]
  metrics:
    - recall_at_10
    - qps
    - latency_p50
    - latency_p95
    - latency_p99
    - peak_rss
    - temporary_disk_bytes
    - build_time
```

14.2 实验顺序

1. 冻结 P0 + B0 + M0，并通过 Baseline Parity Gate。
2. 单模块替换：P1+B0+M0、P0+B1+M0、P0+B0+M1。
3. 强依赖组合：例如同一论文的 P1+B0+M1。
4. 跨论文组合：例如 P1+B2+M3。

   跨论文组合回答的是“不同论文模块是否互补”，不能作为单篇论文复现成功与否的证据。

14.3 两类公平比较

比较方式	控制条件	回答的问题
固定参数	相同 R、L、alpha、数据、线程和硬件	同一参数下，算法机制带来什么变化
固定预算 / Pareto	相同内存、构建时间、索引大小或 Recall 水平	哪个方案在资源-精度-性能权衡上更优

不同论文的参数语义和最优区间可能不同，因此最终结论不能只依赖固定参数比较。

15. 可复现性模式与有效运行判定

15.1 Upstream Faithful

```
reproduction_mode: upstream_faithful
source_modified: false
deterministic_output_required: false
required_records:
  - repository_commit
  - compiler_and_flags
  - dataset_checksum
  - complete_parameters
  - hardware_fingerprint
```

cpp_main 的 merge_shards 使用 random_device 初始化随机数生成器并在截断前 shuffle，因此未修改的原始 Baseline 不保证字节级完全一致。此模式以统计稳定性验证。

代码依据：cpp_main/src/disk_utils.cpp，merge_shards 中 std::random_device、std::mt19937、std::shuffle。

15.2 Instrumented

```
reproduction_mode: instrumented
allowed_changes:
  - stage_timing
  - peak_memory_tracking
  - io_metrics
  - checkpointing
  - artifact_preservation
semantic_changes_allowed: false
parity_gate: required
```

15.3 Deterministic Experimental

```
reproduction_mode: deterministic_experimental
source_modified: true
fixed_seed: 20260731
deviation_record_required: true
logical_graph_equivalence_required: true
```

该模式可以固定随机源，但必须拥有独立 implementation_id，不能与未经修改的原始 Baseline 混为一谈。

15.4 Invalid Run Conditions

- Git commit、数据集 checksum 或参数缺失。
- 工作区存在未记录修改。
- 阶段退出码非零或输出不完整。
- Artifact Manifest 与实际文件不一致。
- LocalToGlobal 映射或节点数量校验失败。
- 查询集、ground truth、距离度量或计时范围与对照组不一致。
- 模块 requires 未被上游 provides 满足。
- 只报告多次运行中的最优一次而未按协议聚合。

16. Harness 与 Loop Engine

16.1 Harness 的职责

```
检查代码与环境
    ↓
构建指定 commit / implementation
    ↓
校验输入 Artifact 与兼容性
    ↓
执行 Partition / Local Build / Merge
    ↓
采集资源和阶段指标
    ↓
验证 Artifact 合法性
    ↓
运行查询评估
    ↓
生成 EvaluationReport 与对比报告
```

Harness 是实验可信度的核心。Spec Kit 负责定义做什么，Harness 负责证明实现和实验是否满足规范。

16.2 Loop Engine 的职责

- 生成和调度参数矩阵。
- 复用已通过校验的上游 Artifact。
- 对可重试失败进行受控重试。
- 维护任务状态、检查点和依赖图。
- 根据固定规则判断继续、终止或进入下一阶段。
- 生成缺失实验清单和异常结果清单。

16.3 Loop 的安全边界

> **禁止指标投机：**Agent 不得通过降低 Recall、跳过查询、改变缓存状态、改变计时范围、只选择最优一次运行或偷偷修改参数来“优化”性能。所有停止与接受条件必须由 experiment-spec.md 和 Harness 校验器定义。

```
loop:
  goal: complete_experiment_matrix
  retry:
    max_attempts: 2
    retry_on:
      - process_crash
      - incomplete_output
      - transient_io_error
  never_retry_on:
      - contract_violation
      - checksum_mismatch
      - incompatible_modules
  stop:
      - all_valid_runs_completed
      - unrecoverable_failure
      - resource_budget_exceeded
```

16.4 Agent 的人工闸门

涉及算法语义修改时，Loop Engine 只能生成候选 patch 和实验建议。进入大数据实验前至少经过：静态检查、单元测试、小图性质测试、SIFT1M 快速验证、人工审查 diff、再运行 SIFT100M 或更大数据集。

17. 推荐目录结构与实施顺序

17.1 完整目录

```
diskann-reproduction/
├── .specify/
│   ├── memory/
│   │   └── constitution.md
│   └── templates/
│       └── paper-reproduction/
│
├── upstream/
│   └── diskann-cpp-main/
│
├── specs/
│   ├── 000-modular-cpp-diskann-pipeline/
│   ├── 001-cpp-main-upstream-baseline/
│   ├── 002-cpp-main-instrumented-baseline/
│   ├── 010-paper-a-partition/
│   ├── 020-paper-b-local-build/
│   ├── 030-paper-c-merge/
│   └── 040-paper-d-coupled-stages/
│
├── contracts/
│   ├── identifiers/
│   ├── artifacts/
│   ├── manifests/
│   ├── capabilities/
│   └── evaluations/
│
├── src/
│   ├── pipeline/
│   ├── partition/
│   │   ├── cpp_main_baseline/
│   │   └── paper_a/
│   ├── local_build/
│   │   ├── cpp_main_baseline/
│   │   └── paper_b/
│   ├── merge/
│   │   ├── cpp_main_baseline/
│   │   └── paper_c/
│   ├── adapters/
│   └── registry/
│
├── harness/
│   ├── build/
│   ├── execute/
│   ├── validate/
│   ├── evaluate/
│   └── compare/
│
├── loop_engine/
│   ├── scheduler/
│   ├── dependency_graph/
│   ├── state_store/
│   ├── retry_policy/
│   ├── experiment_matrix/
│   └── convergence_policy/
```

```
│
└── experiments/
    ├── configs/
    ├── manifests/
    ├── artifacts/
    ├── reports/
    └── results/
```

17.2 实施顺序不是架构删减

完整目标架构保持不变，但实现应按依赖关系推进，避免在基础事实未验证前设计错误接口。该顺序是工程执行顺序，不代表削减功能。

阶段	交付物	退出条件
阶段 0：源码基线	固定 commit、编译环境、原生命令和 Baseline 结果	Baseline 可重复运行并有完整 Manifest
阶段 1：公共契约	Identifier、Artifact、Manifest、Evaluation、Registry	P0+B0+M0 经统一框架通过 parity gate
阶段 2：单模块论文	首个 P1 或 B1 或 M1 Feature	单模块替换、偏差记录和对比闭环完成
阶段 3：多实现组合	requires/provides、Adapter、兼容性检查	至少两个论文实现可安全组合或被正确拒绝
阶段 4：Loop Engine	矩阵调度、复用、重试、状态与报告	长时间实验可断点恢复并保证结果有效性

18. Spec Kit 的实际使用方式

18.1 一次性核心 Feature

```
000 Modular cpp_main Pipeline
└── 定义公共接口、ID 模型、Artifact、Manifest、Evaluation、能力和不变量
```

18.2 一次性 Baseline Feature

```
001 cpp_main Upstream Baseline
└── 固化源码行为、命令、阶段产物、随机性和结果

002 cpp_main Instrumented Baseline
└── 增加观测能力，并通过 parity gate
```

18.3 每篇论文一次

```
010 / 020 / 030 Paper Feature
└──── 定义论文身份、替换范围、算法映射、依赖、偏差和成功标准
```

18.4 每组实验不重写 SDD

```
Experiment YAML
└──── 选择实现、配置参数、数据集、资源预算、复现模式和评估协议
```

18.5 推荐命令流程

```
/speckit.constitution
        ↓
/speckit.specify
        ↓
人工审查 spec.md
        ↓
/speckit.clarify
        ↓
/speckit.plan
        ↓
补充 paper-analysis.md / source-mapping.md / experiment-spec.md
        ↓
/speckit.checklist
        ↓
/speckit.tasks
        ↓
/speckit.analyze
        ↓
/speckit.implement
        ↓
Harness 合约校验与实验验证
        ↓
Loop Engine 批量执行
```

Spec Kit 的输出不能替代论文原文分析、源码映射和实验 Harness。Agent 生成的 Spec、Plan 和 Tasks 必须由人审查，尤其是论文等价性、成功阈值和公平比较条件。

18.6 自定义论文复现模板

当首批两至三篇论文的复现文档稳定后，再将以下字段固化为项目级模板或 Preset：Paper Identity、Replacement Scope、Claimed Mechanism、Source Mapping、Input Assumptions、Output

Guarantees、Reproduction Type、Deviations、Compatibility、Experiment Matrix 和 Paper-specific Checks。

19. 完整验收清单与最终判断

19.1 核心 Pipeline 验收

- 技术基线明确绑定 cpp_main 和固定 commit。
- GlobalNodeId、LocalNodeId、PartitionId 定义清晰。
- 所有局部 Artifact 提供 LocalToGlobalIdMap。
- Artifact、BuildManifest、EvaluationReport 分离。
- 模块通过 ArtifactRef 而不是全量内存对象传递。
- requires/provides 和格式版本在运行前校验。

19.2 Baseline 验收

- P0、B0、M0 的行为与固定 commit 源码一致。
- M0 被准确描述为映射、聚合、去重、shuffle 和 truncate。
- 未将距离剪枝、全局 refinement 或连通性修复误写为原始行为。
- Upstream、Instrumented、Deterministic 三类 Baseline 独立标识。
- 统一框架通过 Baseline Parity Gate。

19.3 论文复现验收

- 论文与官方源码身份明确，commit 可追溯。
- 算法步骤、官方源码和适配代码存在映射。
- 复现类型和所有偏差明确记录。
- 论文特定性质具有独立验证器。
- 结论区分忠实复现、工程适配和性能优劣。

19.4 实验验收

- 先完成 Baseline，再做单模块替换。
- 固定参数与固定预算/Pareto 两类比较同时存在。
- 每次运行有完整环境、配置、输入和输出记录。
- 无效运行不会进入结果聚合。
- 跨论文组合单独报告，不冒充单篇论文复现。

> **最终判断：**修订后的方案既保持了完整工程设计，又与 cpp_main 的实际行为一致。它能够支撑长期论文复现、独立模块替换、耦合方案、跨论文组合、Agent 自动实现与受控实验循环。

附录 A：公共 C++ 接口汇总

```
using GlobalNodeId = uint32_t;
using LocalNodeId  = uint32_t;
using PartitionId  = uint32_t;

struct ArtifactRef {
    std::filesystem::path manifest_path;
    std::string artifact_type;
    std::string format_version;
    std::string checksum;
};

struct StageResult {
    ArtifactRef artifact;
    ArtifactRef build_manifest;
    ArtifactRef stage_report;
};

class PartitionStrategy {
public:
    virtual StageResult run(
        const DatasetRef& dataset,
        const PartitionConfig& config,
        StageContext& context) = 0;
};

class LocalGraphBuilder {
public:
    virtual StageResult build(
        const ArtifactRef& partition_shard,
        const LocalBuildConfig& config,
        StageContext& context) = 0;
};

class GraphMerger {
public:
    virtual StageResult merge(
        const ArtifactRef& partition_manifest,
        std::span<const ArtifactRef> local_graphs,
        const MergeConfig& config,
        StageContext& context) = 0;
};
```

附录 B：推荐的论文 Feature 文件结构

```
010-paper-name-stage/
├── spec.md
├── plan.md
├── research.md
├── paper-analysis.md
```

```
├── source-mapping.md
├── algorithm-mapping.md
├── data-model.md
├── experiment-spec.md
├── compatibility.yaml
├── deviations.md
├── paper-specific-checks.md
├── contracts/
└── tasks.md
```

附录 C：源码行为参考

本设计中关于原始 Baseline 的关键判断应以固定 commit 的官方源码为准，主要参考位置如下：

- microsoft/DiskANN，cpp_main，src/disk_utils.cpp：merge_shards。
- microsoft/DiskANN，cpp_main，src/disk_utils.cpp：build_merged_vamana_index。
- microsoft/DiskANN，cpp_main，include/partition.h：partition_with_ram_budget 等接口。
- microsoft/DiskANN，cpp_main，include/disk_utils.h：merge_shards 与 build_merged_vamana_index 声明。
- microsoft/DiskANN，cpp_main，include/index.h：Index::build、save 与图构建相关接口。

后续固定具体 commit 后，应在 source-mapping.md 中记录精确行号或代码片段 hash。分支内容发生变化时，必须重新完成 Baseline Parity Gate，不能直接沿用旧结果。

End of revised specification